

ENNEURO 技术阶段报告

CUDA C 算子改写

近期工作汇报

EnNeuro 项目组

2026 年 9 月 20 日

文档信息

项目	内容
版本	v1.0
编制方	EnNeuro 项目组
汇报对象	项目组与指导老师
报告日期	2026-09-20
报告范围	CUDA C 算子改写当前阶段
状态	汇报稿

执行摘要

本阶段没有重写整个 EnNeuro 框架，而是在保留 Python 自动求导、CuPy 显存管理和 cuBLAS 矩阵乘法的前提下，把高频、可明确映射到线程的热点算子改写为 CUDA C kernel，并完成 RawModule 路线的正确性和小规模性能闭环。

固定输入算子微基准、MNIST 前向/反向和 1 epoch 训练冒烟测试分别测得约 4.99 倍、19.69 倍和 9.38 倍的 RawModule 加速。正式 DLL 的设备源码已经迁移并构建，但 ctypes/extension 后端尚未接入，完整 MNIST 三方训练也需要在 dtype 修正后重跑。

核心判断

- RawModule 第一版已经证明：自研 kernel 能进入真实 LeNet 前向、反向和参数更新链路。
- 当前数据支持“小规模、已覆盖形状上的明显加速”，不支持对完整训练或所有算子做普遍外推。
- 下一阶段应先完成 extension 闭环和全量复验，再进入 implicit GEMM、split-K、布局与融合优化。

1 目标与问题背景

EnNeuro 原本已经通过 CuPy 使用 GPU。当前瓶颈并不是“没有使用 GPU”，而是一个逻辑算子可能被拆成多次 kernel 启动，并产生用完即丢的中间数组。卷积窗口展开、池化反向的索引搬运和 col2im 都可能增加显存读写和调度开销。

本阶段目标是在不改变 Tensor、Layer、自动求导和 LeNet 公共接口的前提下，用可回退的 CUDA C kernel 替换一部分热点路径；矩阵乘法继续交给 CuPy/cuBLAS，不适配的 dtype、广播或形状继续走 CuPy。

假设与成功标准

项目	定义
技术假设	减少 kernel 启动、临时数组和 Python/CuPy 调度，可降低典型 LeNet 形状的端到端耗时。
正确性标准	与 CuPy/NumPy 对拍，loss 和关键梯度一致；同时检查自研 kernel launch count 非零。
性能标准	warm-up 后用 CUDA Event 计时；RawModule 在当前验证输入上应快于 CuPy。
安全边界	未覆盖的 dtype、广播、dilation 或布局必须可诊断地回退，不得静默给出错误结果。

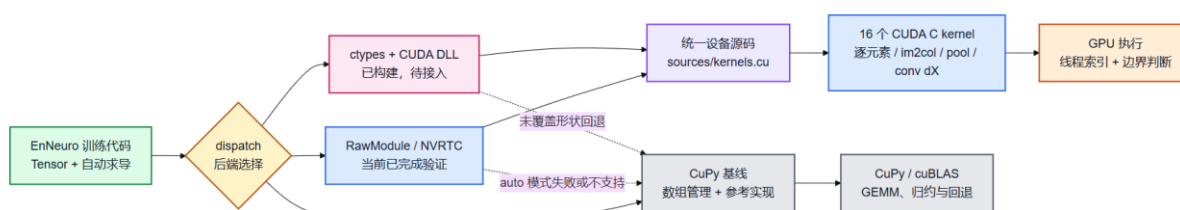


图1 CuPy、RawModule 与 DLL extension 的后端关系

2 技术分析与实现路线

后端分层

路线	作用	当前状态
CuPy	功能基线、回退、显存管理和 GEMM/归约	保留
RawModule / NVRTC	运行时编译统一 .cu 源码，快速验证索引、梯度与性能	第一轮闭环已完成
ctypes + CUDA DLL	正式二进制交付和稳定 ABI	DLL 已构建， loader 待接入

算子线程映射

- 逐元素算子：一个线程对应一个线性元素，统一用 $i < n$ 处理尾部。
- 卷积前向：CUDA im2col + CuPy/cuBLAS GEMM + CUDA bias。
- 卷积输入梯度：一个线程负责一个 gx 元素，gather 所有关联的 $gy \times W$ ，不生成完整 $gcol$ 。
- 池化前向：一个线程扫描一个输出窗口并保存 argmax ；池化反向当前使用 $\text{scatter} + \text{atomicAdd}$ 。

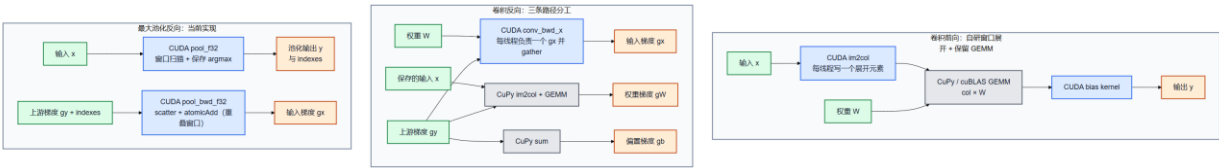


图2 卷积与池化中自研CUDA C和CuPy/cuBLAS的分工

3 已完成工作

模块	已经完成的内容	证据
统一设备源码	RawModule 与 DLL 共用 sources/kernels.cu，避免公式分叉	16 个 __global__ kernel
编译与缓存	按源码哈希、计算能力和选项缓存 RawModule；缓存函数句柄	compiler.py / dispatch.py
算子接入	基础逐元素、ReLU、bias、im2col、Conv2d dX、MaxPool 前后向	core.py / functions.py
实验公平性	修复权重 dtype 提升和 Adam 跨实例共享状态	module.py / optim.py
可观测性	记录实际后端、回退原因、错误和每个 kernel 发射次数	launch_counts / diagnostics
DLL 路线	完成 C ABI、host launcher、CUDA 12.6 sm_89 构建	api.h / library.cu / manifest

当前 16 个 CUDA C kernel

实际实现包括 add、sub、mul、div；neg、exp、log、pow、relu、relu_bwd、sigmoid；bias；im2col；pool 前向、pool 反向；以及卷积输入梯度 conv_bwd_x。矩阵乘法、卷积 gW/gb 和部分归约仍由 CuPy/cuBLAS 完成。

正确性验证

阶段验收报告记录了 CUDA 12.6 环境下 12 项 CUDA kernel/LeNet 回归测试通过。覆盖逐元素、激活、bias、im2col、卷积前向与梯度、池化索引与重叠窗口梯度，以及 LeNet 端到端前向/反向；同时检查 conv_bwd_x_f32、pool_bwd_f32 等关键 kernel 确实发射。

这组 12 passed 是项目报告中的历史运行记录。正式提交前应在清洁环境中重跑并归档完整日志，以排除依赖和驱动变化。

DLL 构建边界

面向 sm_89 的 enneuro_cuda_sm89.dll 已生成，ABI 符号可加载；但 dispatch 的 extension 选项尚未绑定 ctypes loader。因此当前应表述为“DLL 构建完成”，不能表述为“框架已经支持 DLL 后端”。

4 当前实测结果

以下数据来自 artifacts/cuda_stage1_probe 的优化后 JSON。三组实验均在同一 sm_89 设备、CuPy 13.3.0、CUDA 12.6 环境中进行。

场景	CuPy	RawModule	结果
固定输入算子微基准	0.91969 ms	0.18432 ms	4.99×; 最大绝对误差 1.1444e-05
MNIST 512 张, 前向+反向	76.04 ms/batch	3.86 ms/batch	19.69×; loss 与 accuracy 相同
MNIST 512/512, 1 epoch Adam	716.89 ms	76.46 ms	9.38×; loss 相同, accuracy 差 0.20 个百分点

CUDA C RawModule 与 CuPy: 当前实测耗时

同一设备、固定小规模输入; 数值越短越快, 右侧标注为 RawModule 相对 CuPy 的加速倍数

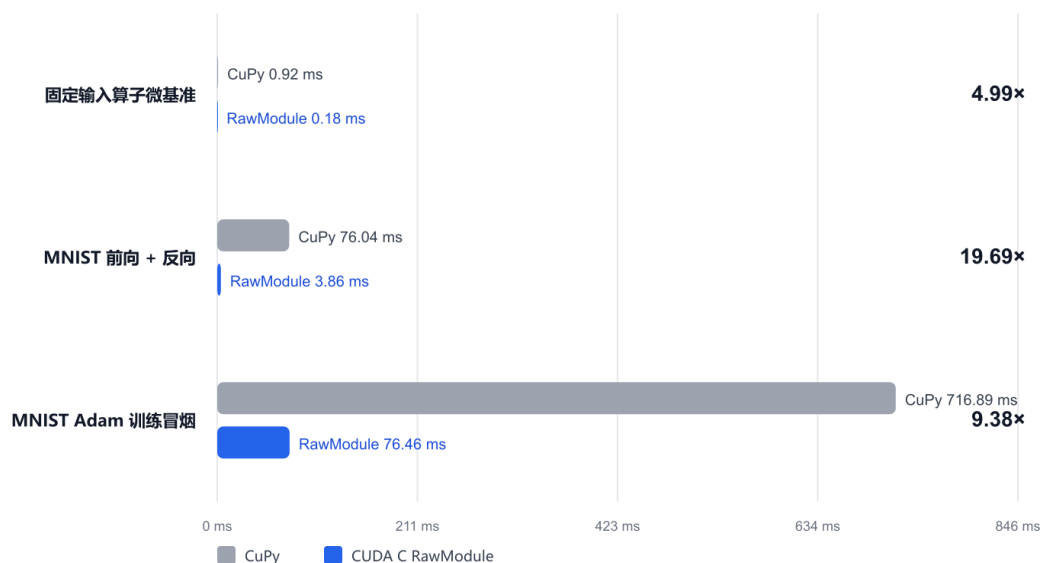


图3 当前三个小规模场景的 CuPy 与 RawModule 耗时

结果解读

- 固定输入微基准表明组合算子端到端耗时下降, 不只是单个 kernel 的孤立结果。
- 512 张 MNIST 的前向/反向中, mean loss 均为 2.2972087, 准确率均为 6.25%; 这是未训练模型的一致性检查, 不是模型效果结论。
- 1 epoch 冒烟测试的训练 loss 均为 2.1758721, 测试准确率为 56.64% 和 56.84%; 样本和 epoch 太小, 不能外推完整训练。

- 8 个 batch 中, im2col、bias、ReLU、pool 前后向和 conv_bwd_x 均有非零 launch count, 证明没有静默回退。

5 解释、限制与决策

当前证据支持的判断

- 在已覆盖的 float32、连续数组和典型 LeNet 形状上，自研 CUDA C 可以进入现有自动求导链路。
- 当前数值对拍和小规模训练闭环成立，且实测显示明显耗时收益。
- 统一 kernels.cu 已经同时支撑 RawModule 和 DLL 构建，为正式 extension 接入提供稳定基础。

当前证据不支持的外推

限制	影响
规模限制	4.99×、19.69×、9.38×不能直接外推到完整 MNIST 或其他网络。
覆盖限制	无独立 col2im、自研 GEMM、完整 Softmax/CrossEntropy、sum/mean/broadcast kernel；gW/gb 仍为 CuPy。
输入限制	非 float32、广播、dilation≠1 或不满足布局条件时仍可能回退。
后端限制	DLL 可构建、可加载 ABI，但 extension 尚未接入框架。
确定性限制	池化反向是 scatter + atomicAdd，重叠窗口的浮点加法顺序可能变化。
历史结果限制	优化前旧三方 MNIST 结果受 dtype 问题影响，不用于支持本轮结论。

阶段决策

建议将 RawModule 路线认定为“阶段验证通过、需要复验归档”，进入 DLL extension 接入和完整三方训练阶段；暂不宣称全量算子改写完成，也不把小规模加速比作为最终性能指标。

6 后续计划与验收标准

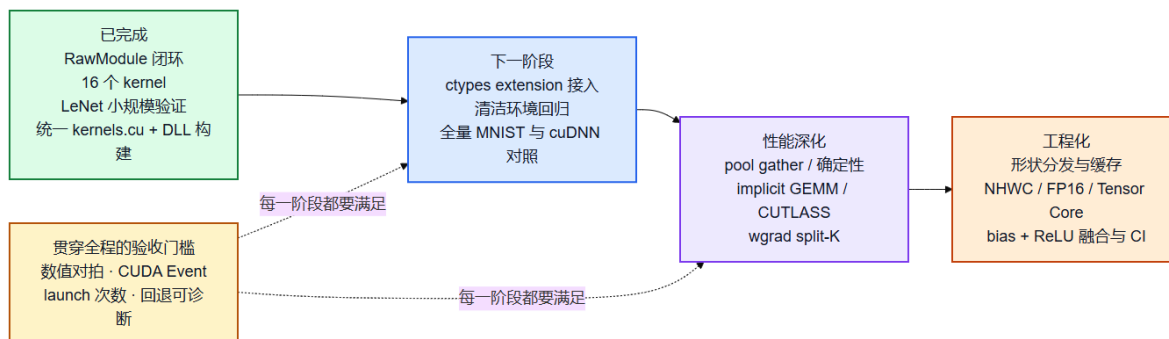


图 4 从当前RawModule 闭环到工程化后端的路线图

优先级	工作	验收标准
P0	清洁环境重跑 12 项回归；全量 MNIST 三方复验；增加 cupyx.cudnn 基线	数值、launch count、CUDA Event 和依赖版本可复核
P1	完成 ctypes loader 与 extension dispatch；处理 ABI、stream、错误码和自动回退	CuPy/RawModule/extension 同输入对拍通过
P2	补齐形状、padding/stride、dilation、dtype、广播和确定性覆盖矩阵	所有未覆盖场景有明确回退和诊断
P3	评估 pool gather、implicit GEMM/CUTLASS、wgrad split-K、NHWC 和低精度	按形状实测选择，不优于基线的 kernel 不进入默认路径
P4	建立 CI、架构矩阵、二进制 hash/编译参数追踪和性能回归监测	源码、DLL、环境与报告结果可追溯

近期执行顺序

1. 先修复和固化测试环境，重跑当前 RawModule 结果。
2. 随后完成 extension loader，用同一测试集做三方数值与性能对比。
3. 全量训练结果稳定后，再决定哪些热点值得进入 implicit GEMM、split-K 或融合优化。

7 PPT 页面拆解建议

文稿可直接拆成 10 页汇报。每页只保留一个结论，性能数字和结果边界放在同一页或相邻页，避免把小规模结果误解为最终训练结论。

页码	主题	核心内容	素材
1	工作目标与结论	为什么改、当前完成到哪一步	执行摘要
2	CuPy 路径瓶颈	kernel 启动、中间数组、调度开销	问题背景
3	三层后端路线	CuPy、RawModule、DLL 的分工	图 1
4	算子实现边界	自研 kernel 与 cuBLAS/CuPy 的分工	图 2
5	已经完成的代码	16 个 kernel、缓存、dtype/Adam 修正	完成工作表
6	正确性证据	历史 12 项回归、误差、launch count	验证小节
7	性能结果	4.99×、19.69×、9.38×	图 3
8	结果边界	小规模、未全量、DLL 未接入、atomicAdd	限制表
9	下一阶段路线	extension、全量训练、cuDNN、性能深化	图 4
10	决策与资源	进入 Po/P1，明确验收标准	阶段决策

建议收尾话术

当前阶段已经完成“可验证的 CUDA C 第一版”，还没有完成“覆盖所有算子和所有形状的最终后端”。下一步先完成 DLL extension 闭环与完整训练复验，再针对实测热点推进 gather、implicit GEMM、split-K 和融合。每一次改写都必须回答三个问题：算得对不对、真的走没走、是否值得保留。

附录 证据与来源

证据	路径或说明
阶段验收记录	doc/CUDA_C_阶段验收汇报.md
后端分派	code/eneuro/base/cuda/dispatch.py
设备 kernel	code/eneuro/base/cuda/sources/kernels.cu
RawModule 缓存	code/eneuro/base/cuda/compiler.py
DLL ABI	code/eneuro/base/cuda/sources/api.h、library.cu
正确性测试	code/tests/test_cuda_kernels.py、test_cuda_lenet.py
微基准	artifacts/cuda_stage1_probe/optimized_cuda_gpu_two_way.json
MNIST 前向/反向	artifacts/cuda_stage1_probe/optimized_mnist_gpu_two_way.json
训练冒烟	artifacts/cuda_stage1_probe/optimized_mnist_training_gpu_two_way.json

汇报时需要主动说明的三条边界

- 当前速度数据来自固定小规模输入，不等价于完整训练最终加速比。
- DLL 已经构建但尚未成为框架可调用的 extension 后端。
- 当前实现是 CUDA C 与 CuPy/cuBLAS 的混合后端，不是所有计算都由自研 kernel 完成。